Guide d'implémentation — Modélisation de la diffusion tumorale

> **Langage :** Python 3.10+
> **Niveau supposé :** intermédiaire (numpy, matplotlib connus)
> **Objectif final :** simuler et visualiser en 3D la propagation d'un gliome sur un cerveau BraTS

Sommaire

1. Vue d'ensemble du pipeline
2. Librairies nécessaires
3. Étape 1 — Chargement des données IRM
4. Étape 2 — Prétraitement et construction des masques
5. Étape 3 — Construction de la carte de diffusion D(x)
6. Étape 4 — Construction de l'opérateur Laplacien
7. Étape 5 — Boucle de simulation (schéma de Crank-Nicolson)
8. Étape 6 — Visualisation 3D
9. Fichier principal `main.py`
10. Organisation des fichiers

---

1. Vue d'ensemble du pipeline

Voici l'enchaînement logique des étapes, du fichier IRM brut jusqu'à la visualisation :

```
Fichier NIfTI (.nii.gz)
      |
      ▼
 [load_nifti()]         ← lecture du fichier IRM
      |
      ▼
 [preprocess()]         ← nettoyage, normalisation, extraction du cerveau
      |
      ▼
 [build_tissue_masks()]  ← séparer matière blanche / grise / LCR
      |
      ▼
 [build_diffusion_map()] ← assigner D_w ou D_g à chaque voxel
      |
      ▼
 [build_laplacian()]     ← construire l'opérateur de diffusion (matrice creuse)
      |
      ▼
 [init_tumor()]          ← poser la condition initiale c(x,0)
      |
      ▼
 [run_simulation()]      ← boucle temporelle Crank-Nicolson
      |
      ▼
 [visualize_3d()]        ← rendu volumique ou isosurface 3D
```

Chaque étape correspond à un fichier Python distinct. On gardera les fonctions courtes et testables séparément.

---

2. Librairies nécessaires

```bash
pip install nibabel numpy scipy matplotlib
```

Librairie	Utilité	Niveau de complexité
`nibabel`	Lire les fichiers IRM au format NIfTI (.nii.gz)	Facile — 2-3 fonctions à connaître
`numpy`	Manipuler les grilles 3D de voxels	Familier
`scipy.sparse`	Stocker le Laplacien creux (économie mémoire critique en 3D)	Moyen — doc ci-dessous
`scipy.sparse.linalg`	Résoudre le système linéaire à chaque pas de temps	Moyen — une seule fonction
`matplotlib`	Visualisation 2D (coupes, courbes)	Familier

Documentation à garder sous la main :

- `scipy.sparse` : https://docs.scipy.org/doc/scipy/reference/sparse.html
- `nibabel` quickstart : https://nipy.org/nibabel/gettingstarted.html

> **Pourquoi `scipy.sparse` ?** Un cerveau BraTS fait environ 240×240×155 voxels = ~9 millions de voxels. Le Laplacien 3D est une matrice 9M × 9M. En dense, c'est ~650 Go de RAM. En creux (sparse), c'est ~500 Mo — gérable sur un laptop standard.

---

3. Étape 1 — Chargement des données IRM

Fichier : `src/loader.py`

Les fichiers BraTS sont au format **NIfTI** (`.nii` ou `.nii.gz`). Chaque fichier contient un tableau 3D de valeurs d'intensité ainsi qu'une matrice affine qui relie les coordonnées voxel aux coordonnées réelles en millimètres.

Pour chaque patient BraTS, vous aurez quatre modalités IRM :

Fichier	Modalité	Utilité dans notre modèle
`*_t1.nii.gz`	T1 natif	Anatomie générale
`*_t1ce.nii.gz`	T1 + gadolinium	Zone tumorale active (rehaussement)
`*_t2.nii.gz`	T2	Œdème péritumoral
`*_flair.nii.gz`	FLAIR	Envahissement tumoral diffus
`*_seg.nii.gz`	Segmentation	Masque tumoral de référence (vérité terrain)

python

```python
# src/loader.py

import nibabel as nib
import numpy as np


def load_nifti(filepath: str) -> tuple[np.ndarray, np.ndarray]:
    """
    Charge un fichier IRM au format NIfTI.

    Paramètres
    ----------
    filepath : str
        Chemin vers le fichier .nii ou .nii.gz

    Retourne
    --------
    data : np.ndarray, shape (X, Y, Z)
        Tableau 3D des intensités de voxels
    affine : np.ndarray, shape (4, 4)
        Matrice affine voxel → coordonnées réelles (mm)
        Utile pour connaître la taille réelle d'un voxel (voxel_size)

    Notes
    -----
    La taille d'un voxel en mm est récupérable via :
        nib.load(filepath).header.get_zooms()
    Pour BraTS : généralement (1.0, 1.0, 1.0) mm isotrope
    """
    pass


def get_voxel_size(filepath: str) -> tuple[float, float, float]:
    """
    Retourne les dimensions physiques d'un voxel en millimètres.

    Paramètres
    ----------
    filepath : str
        Chemin vers le fichier NIfTI

    Retourne
    --------
    tuple (dx, dy, dz) en mm
        Pour BraTS : généralement (1.0, 1.0, 1.0)
```

```
    Pour BraTS, typiquement (1.0, 1.0, 1.0)

    Notes
    -----
    Ces valeurs servent à calibrer les coefficients de diffusion D.
    Un voxel de 2mm nécessite de diviser D par 4 dans la discrétisation.
    """
    pass
```

---

4. Étape 2 — Prétraitement et construction des masques

Fichier : `src/preprocessing.py`

Avant de simuler, il faut préparer trois choses :

1. **Normaliser** les intensités IRM (les valeurs brutes varient selon les scanners)
2. **Extraire le cerveau** (skull stripping) — BraTS fournit des images déjà skull-strippées, ce qui simplifie cette étape
3. **Construire les masques** de tissu pour affecter les bons coefficients D

La segmentation des tissus peut se faire simplement par **seuillage des intensités T1** (approche classique). Une approche plus robuste utiliserait FSL ou FreeSurfer, mais pour commencer le seuillage suffit.

python

```python
# src/preprocessing.py

import numpy as np


def normalize_volume(volume: np.ndarray) -> np.ndarray:
    """
    Normalise un volume IRM entre 0 et 1.

    Méthode recommandée : normalisation par les percentiles 1 et 99
    (résistante aux valeurs aberrantes en périphérie du crâne).

    Paramètres
    ----------
    volume : np.ndarray, shape (X, Y, Z)
        Volume IRM brut

    Retourne
    --------
    np.ndarray, shape (X, Y, Z), valeurs dans [0, 1]

    Notes
    -----
    Utiliser np.percentile(volume, 1) et np.percentile(volume, 99)
    pour les bornes de normalisation.
    """
    pass


def extract_brain_mask(volume: np.ndarray, threshold: float = 0.01) -> np.ndarray:
    """
    Construit un masque binaire du cerveau (1 = cerveau, 0 = fond/crâne).

    Pour BraTS (déjà skull-strippé), un simple seuil sur le volume
    normalisé suffit. Les voxels > threshold appartiennent au cerveau.

    Paramètres
    ----------
    volume : np.ndarray
        Volume normalisé (entre 0 et 1)
    threshold : float
        Seuil d'intensité en dessous duquel un voxel est considéré hors cerveau

    Retourne
```

```
    --------
    mask : np.ndarray booléen, shape (X, Y, Z)
        True là où il y a du tissu cérébral

    Notes
    -----
    Pour fermer les petits trous dans le masque, on peut appliquer
    une opération morphologique : scipy.ndimage.binary_fill_holes(mask)
    Doc : https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.binary_fill_holes.html
    """
    pass


def build_tissue_masks(
    t1_volume: np.ndarray,
    brain_mask: np.ndarray
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    Sépare le cerveau en trois zones par seuillage d'intensité T1.

    Approche par seuillage à deux niveaux :
    - Les voxels T1 très brillants  → matière blanche (WM)
    - Les voxels T1 moyennement brillants → matière grise (GM)
    - Les voxels T1 sombres (dans le masque) → LCR / autres

    Paramètres
    ----------
    t1_volume : np.ndarray
        Volume T1 normalisé
    brain_mask : np.ndarray booléen
        Masque du cerveau (issu de extract_brain_mask)

    Retourne
    --------
    wm_mask : np.ndarray booléen  — matière blanche
    gm_mask : np.ndarray booléen  — matière grise
    csf_mask : np.ndarray booléen — LCR (liquide céphalo-rachidien)

    Notes
    -----
    Seuils indicatifs sur T1 normalisé (à ajuster selon les données) :
        WM  : intensité > 0.75
        GM  : 0.40 < intensité ≤ 0.75
        CSF : intensité ≤ 0.40 (dans le masque cerveau)

    Ces seuils peuvent être affinés avec un algorithme de clustering
    (k-means sur les intensités, k=3) si les résultats ne sont pas satisfaisants.
```

```
    Voir : sklearn.cluster.KMeans — mais rester sur le seuillage pour commencer.
    """
    pass


def get_initial_tumor(seg_volume: np.ndarray, label: int = 1) -> np.ndarray:
    """
    Extrait la condition initiale c(x,0) depuis la segmentation BraTS.

    Les labels BraTS standard sont :
        0 → fond / tissu sain
        1 → tumeur nécrotique (non-rehaussante)
        2 → œdème péritumoral
        4 → tumeur active (rehaussante)

    Paramètres
    ----------
    seg_volume : np.ndarray, shape (X, Y, Z)
        Volume de segmentation BraTS
    label : int
        Label à extraire (par défaut 1, cœur tumoral)
        Passer label=None pour prendre tous les labels tumoraux (1, 2, 4)

    Retourne
    --------
    c0 : np.ndarray float, shape (X, Y, Z), valeurs dans {0.0, 1.0}
        Densité initiale : 1.0 dans la région tumorale, 0.0 ailleurs

    Notes
    -----
    On peut lisser c0 avec un filtre gaussien léger pour éviter des
    discontinuités numériques au bord de la tumeur initiale :
        scipy.ndimage.gaussian_filter(c0, sigma=1.0)
    """
    pass
```

---

5. Étape 3 — Construction de la carte de diffusion D(x)

Fichier : `src/diffusion_map.py`

Cette étape construit le tableau 3D qui associe à chaque voxel son coefficient de diffusion D, selon qu'il appartient à la matière blanche ou grise.

python

```python
# src/diffusion_map.py

import numpy as np

# Constantes biologiques du modèle de Swanson
D_WHITE_MATTER = 0.013   # mm²/jour — matière blanche
D_GRAY_MATTER  = 0.0013  # mm²/jour — matière grise
D_CSF          = 0.0     # LCR : pas de migration tumorale


def build_diffusion_map(
    wm_mask: np.ndarray,
    gm_mask: np.ndarray,
    csf_mask: np.ndarray,
    d_white: float = D_WHITE_MATTER,
    d_gray: float = D_GRAY_MATTER,
    d_csf: float = D_CSF
) -> np.ndarray:
    """
    Construit la carte spatiale D(x) du coefficient de diffusion.

    Assigne à chaque voxel sa valeur de D selon le type de tissu.
    Le résultat est un tableau 3D de même forme que les masques.

    Paramètres
    ----------
    wm_mask, gm_mask, csf_mask : np.ndarray booléen, shape (X, Y, Z)
        Masques de tissu issus de build_tissue_masks()
    d_white, d_gray, d_csf : float
        Coefficients de diffusion en mm²/jour

    Retourne
    --------
    D_map : np.ndarray float, shape (X, Y, Z)
        Carte de diffusion : D_map[i,j,k] = coefficient D du voxel (i,j,k)

    Notes
    -----
    Voxels hors cerveau (hors des trois masques) : D = 0.
    Exemple d'utilisation :
        D_map = build_diffusion_map(wm, gm, csf)
        # D_map[i,j,k] vaut D_WHITE_MATTER si wm[i,j,k] est True, etc.
    """
    pass
```

```python
def rescale_diffusion_for_voxel_size(
    D_map: np.ndarray,
    voxel_size_mm: float
) -> np.ndarray:
    """
    Corrige la carte D en tenant compte de la taille physique des voxels.

    La discrétisation du Laplacien divise par h² (taille du voxel²).
    Si les voxels ne font pas 1mm, il faut ajuster D en conséquence.

    Paramètres
    ----------
    D_map : np.ndarray
        Carte de diffusion en mm²/jour (issu de build_diffusion_map)
    voxel_size_mm : float
        Taille d'un voxel en mm (h dans la discrétisation)
        Pour BraTS : 1.0 mm, donc cette fonction ne change rien

    Retourne
    --------
    D_map_scaled : np.ndarray
        Carte de diffusion adimensionnée pour la grille numérique

    Notes
    -----
    La relation est : D_numérique = D_physique / h²
    Pour BraTS (h=1mm), D_numérique = D_physique.
    """
    pass
```

---

6. Étape 4 — Construction de l'opérateur Laplacien

Fichier : `src/laplacian.py`

C'est l'étape la plus technique. Le Laplacien discret 3D $\nabla\cdot(D\nabla c)$ est représenté sous forme de **matrice creuse** (sparse matrix). Chaque voxel est un nœud connecté à ses 6 voisins (haut/bas, gauche/droite, avant/arrière).

La discrétisation par différences finies donne, pour un voxel intérieur (i, j, k) :

$$
\begin{aligned}
[\nabla\cdot(D\nabla c)]_{i,j,k} \approx\ & D_{i+½}\,(c_{i+1,j,k} - c_{i,j,k}) - D_{i-½}\,(c_{i,j,k} - c_{i-1,j,k}) \\
& + D_{j+½}\,(c_{i,j+1,k} - c_{i,j,k}) - D_{j-½}\,(c_{i,j,k} - c_{i,j-1,k}) \\
& + D_{k+½}\,(c_{i,j,k+1} - c_{i,j,k}) - D_{k-½}\,(c_{i,j,k} - c_{i,j,k-1})
\end{aligned}
$$

où $D_{i+½} = (D_{i,j,k} + D_{i+1,j,k}) / 2$ (moyenne aux interfaces)

python

```python
# src/laplacian.py

import numpy as np
from scipy.sparse import lil_matrix, csr_matrix


def flatten_index(i: int, j: int, k: int, shape: tuple) -> int:
    """
    Convertit des indices 3D (i,j,k) en indice 1D pour la matrice creuse.

    Le vecteur d'état c est stocké comme un vecteur 1D de longueur X*Y*Z.
    Cette fonction donne la correspondance entre position 3D et index 1D.

    Paramètres
    ----------
    i, j, k : int
        Indices 3D du voxel
    shape : tuple (X, Y, Z)
        Dimensions du volume

    Retourne
    --------
    idx : int
        Index 1D correspondant

    Notes
    -----
    Équivalent à : np.ravel_multi_index((i, j, k), shape)
    Ou à la main : idx = i * Y * Z + j * Z + k
    """
    pass


def build_laplacian_3d(
    D_map: np.ndarray,
    brain_mask: np.ndarray
) -> csr_matrix:
    """
    Construit le Laplacien discret 3D avec diffusion hétérogène.

    Crée une matrice creuse L de taille (N x N) où N = nombre de voxels
    dans le masque cerveau. L encode l'opérateur ∇·(D∇c) discrétisé
    par différences finies centrées sur grille régulière (h=1 voxel).

    Paramètres
```

```
    Paramètres
    ----------
    D_map : np.ndarray, shape (X, Y, Z)
        Carte du coefficient de diffusion par voxel
    brain_mask : np.ndarray booléen, shape (X, Y, Z)
        Seuls les voxels dans le masque sont inclus dans la matrice

    Retourne
    --------
    L : scipy.sparse.csr_matrix, shape (N, N)
        Matrice du Laplacien discret en format CSR (optimal pour la résolution)

    Notes
    -----
    Stratégie de construction recommandée :
    1. Créer une matrice lil_matrix (facile à remplir ligne par ligne)
    2. Pour chaque voxel (i,j,k) dans le masque :
       - Calculer les coefficients D aux 6 interfaces (moyenne des voisins)
       - Remplir la diagonale et les 6 entrées hors-diagonale
       - Ignorer les voisins hors masque (condition de Neumann)
    3. Convertir en csr_matrix pour la résolution (csr est bien plus rapide)

    Condition de Neumann (∂c/∂n = 0 au bord) : si un voisin est hors
    masque, on ne l'ajoute tout simplement pas à la matrice (flux = 0).

    Doc scipy.sparse.lil_matrix :
        https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.lil_matrix.html
    Doc scipy.sparse.csr_matrix :
        https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csr_matrix.html

    ⚠️ Attention performances : cette boucle peut être lente en Python pur
    sur un cerveau complet (~9M voxels). Travailler d'abord sur un sous-volume
    réduit (ex: 50×50×50) pour tester, puis sur le volume complet.
    """
    pass
```

---

7. Étape 5 — Boucle de simulation (schéma de Crank-Nicolson)

Fichier : `src/simulation.py`

Le schéma de **Crank-Nicolson** est le schéma numérique recommandé pour cette EDP. C'est une moyenne du schéma d'Euler explicite (instable si dt trop grand) et implicite (stable mais diffusif). Il est **stable inconditionnellement** pour la diffusion et d'ordre 2 en temps.

L'équation à résoudre à chaque pas de temps est :

$$(I - dt/2 \cdot L) \cdot c^{n+1} = (I + dt/2 \cdot L) \cdot c^n + dt \cdot F(c^n)$$

où :

- L = matrice du Laplacien (construite à l'étape précédente)
- c^n = vecteur de densité au temps t_n (aplati en 1D)
- c^{n+1} = vecteur de densité au temps t_{n+1}
- $F(c^n)$ = terme de prolifération logistique : $\rho \cdot c^n \cdot (1 - c^n)$
- I = matrice identité

C'est un **système linéaire** qu'on résout à chaque pas de temps.

python

```python
# src/simulation.py

import numpy as np
from scipy.sparse import eye, csr_matrix
from scipy.sparse.linalg import spsolve


def build_cn_matrices(
    L: csr_matrix,
    dt: float
) -> tuple[csr_matrix, csr_matrix]:
    """
    Pré-calcule les deux matrices du schéma Crank-Nicolson.

    Ces matrices sont constantes (ne dépendent pas de c ni de t),
    donc on les construit une seule fois avant la boucle temporelle.

    Paramètres
    ----------
    L : csr_matrix, shape (N, N)
        Matrice du Laplacien
    dt : float
        Pas de temps en jours

    Retourne
    --------
    A : csr_matrix — matrice du membre gauche  : (I - dt/2 · L)
    B : csr_matrix — matrice du membre droit   : (I + dt/2 · L)

    Notes
    -----
    scipy.sparse.eye(N) crée l'identité creuse de taille N.
    Les opérations +/- entre matrices creuses sont directement supportées.
    Doc spsolve : https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.spsolve.html
    """
    pass


def proliferation_term(c: np.ndarray, rho: float) -> np.ndarray:
    """
    Calcule le terme de prolifération logistique F(c) = ρ · c · (1 − c).

    Paramètres
    ----------
```

```
    c : np.ndarray, shape (N,)
        Vecteur de densité tumorale (valeurs dans [0, 1])
    rho : float
        Taux de prolifération en j⁻¹ (typiquement 0.012 à 0.025)

    Retourne
    --------
    F : np.ndarray, shape (N,)
        Terme source de prolifération pour chaque voxel

    Notes
    -----
    C'est la fonction la plus simple du pipeline : rho * c * (1 - c)
    Le clipping np.clip(c, 0, 1) avant le calcul évite les légères
    instabilités numériques qui feraient sortir c de [0,1].
    """
    pass


def run_simulation(
    c0: np.ndarray,
    A: csr_matrix,
    B: csr_matrix,
    rho: float,
    dt: float,
    n_steps: int,
    brain_mask: np.ndarray,
    save_every: int = 10
) -> list[np.ndarray]:
    """
    Lance la simulation temporelle par le schéma de Crank-Nicolson.

    À chaque pas de temps :
    1. Calculer le membre droit : rhs = B · c^n + dt · F(c^n)
    2. Résoudre le système : A · c^{n+1} = rhs  → spsolve(A, rhs)
    3. Clipper c dans [0,1] pour éviter les dépassements numériques
    4. Sauvegarder le snapshot si nécessaire

    Paramètres
    ----------
    c0 : np.ndarray, shape (N,)
        Condition initiale aplatie en 1D (N = nombre de voxels dans masque)
    A, B : csr_matrix
        Matrices Crank-Nicolson (issues de build_cn_matrices)
    rho : float
        Taux de prolifération (j⁻¹)
    dt : float
```

```
        Pas de temps (jours). Recommandé : 0.5 à 1.0 jour.
    n_steps : int
        Nombre total de pas de temps
    brain_mask : np.ndarray booléen, shape (X, Y, Z)
        Nécessaire pour reconstruire les volumes 3D pour la sauvegarde
    save_every : int
        Fréquence de sauvegarde des snapshots (tous les N pas de temps)

    Retourne
    --------
    snapshots : list of np.ndarray, shape (X, Y, Z) chacun
        Liste des volumes 3D sauvegardés à intervalles réguliers
        Le premier élément est c0 reconstruit en 3D.

    Notes
    -----
    Le temps de simulation total est : dt * n_steps jours.
    Pour simuler 300 jours avec dt=1 : n_steps=300, dt=1.0.
    Afficher une barre de progression : voir tqdm (pip install tqdm),
    ou simplement print() tous les 50 pas si on veut rester simple.
    """
    pass


def unflatten_volume(
    c_flat: np.ndarray,
    brain_mask: np.ndarray
) -> np.ndarray:
    """
    Reconstruit un volume 3D depuis le vecteur 1D de simulation.

    Le vecteur c_flat ne contient que les voxels dans le masque cerveau.
    Cette fonction le remet dans la grille 3D originale.

    Paramètres
    ----------
    c_flat : np.ndarray, shape (N,)
        Vecteur de densité (N = nombre de voxels True dans brain_mask)
    brain_mask : np.ndarray booléen, shape (X, Y, Z)

    Retourne
    --------
    volume : np.ndarray float, shape (X, Y, Z)
        Volume 3D reconstruit (voxels hors masque = 0)

    Notes
```

```
    -----
    Technique recommandée :
        volume = np.zeros(brain_mask.shape)
        volume[brain_mask] = c_flat
    C'est tout — l'indexation booléenne de numpy fait le travail.
    """
    pass
```

---

8. Étape 6 — Visualisation 3D

Fichier : `src/visualization.py`

Deux approches accessibles pour la 3D, de la plus simple à la plus visuelle :

Option A — Coupes 2D animées (le plus simple, recommandé pour commencer)
Afficher les coupes axiales/sagittales/coronales à différents temps de simulation.

Option B — Isosurface 3D avec matplotlib
Extraire la surface c = seuil (par exemple c = 0.1) et l'afficher en 3D avec `matplotlib`. L'algorithme **Marching Cubes** (disponible dans `scikit-image`) fait cette extraction.

```bash
pip install scikit-image
```

python

```python
# src/visualization.py

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection


def plot_axial_slices(
    snapshots: list[np.ndarray],
    slice_idx: int = None,
    times_days: list[float] = None,
    cmap: str = "hot"
) -> None:
    """
    Affiche une grille de coupes axiales (vue du dessus) à différents temps.

    Paramètres
    ----------
    snapshots : list of np.ndarray, shape (X, Y, Z) chacun
        Volumes 3D issus de run_simulation()
    slice_idx : int, optionnel
        Index de la coupe axiale à afficher (selon l'axe Z)
        Si None, prend la coupe médiane automatiquement
    times_days : list of float, optionnel
        Liste des temps en jours correspondant aux snapshots (pour le titre)
    cmap : str
        Colormap matplotlib. 'hot' ou 'inferno' sont lisibles pour une densité.

    Notes
    -----
    plt.subplot() pour mettre les coupes côte à côte.
    volume[:, :, slice_idx] donne la coupe axiale à l'index slice_idx.
    """
    pass


def plot_3d_isosurface(
    volume: np.ndarray,
    iso_level: float = 0.15,
    brain_mask: np.ndarray = None,
    title: str = "Simulation tumorale — isosurface 3D"
) -> None:
    """
    Affiche une isosurface 3D de la densité tumorale avec matplotlib.
```

```
    Utilise l'algorithme Marching Cubes (skimage.measure.marching_cubes)
    pour extraire la surface c = iso_level, puis l'affiche avec
    mpl_toolkits.mplot3d.

    Paramètres
    ----------
    volume : np.ndarray, shape (X, Y, Z)
        Volume de densité tumorale à un instant donné
    iso_level : float
        Niveau de densité pour l'isosurface (0.15 = frontière d'invasion)
        Un niveau bas (0.05–0.15) montre l'invasion diffuse.
        Un niveau haut (0.5–0.8) montre le cœur tumoral.
    brain_mask : np.ndarray booléen, optionnel
        Si fourni, affiche aussi un contour translucide du cerveau pour
        donner le contexte anatomique
    title : str
        Titre de la figure

    Notes
    -----
    Algorithme Marching Cubes :
        from skimage.measure import marching_cubes
        verts, faces, normals, values = marching_cubes(volume, level=iso_level)
    Doc : https://scikit-image.org/docs/stable/api/skimage.measure.html#skimage.measure.marching_cubes

    Affichage avec matplotlib 3D :
        fig = plt.figure()
        ax = fig.add_subplot(111, projection='3d')
        mesh = Poly3DCollection(verts[faces])
        mesh.set_alpha(0.7)
        ax.add_collection3d(mesh)

    ⚠️ Sur un volume plein résolution (240×240×155), marching_cubes
    peut être lent (~10-30s). Recommandé : downsampler le volume d'abord
    (voir downsample_volume ci-dessous) ou travailler sur une ROI.
    """
    pass


def plot_3d_evolution(
    snapshots: list[np.ndarray],
    iso_level: float = 0.15,
    times_days: list[float] = None
) -> None:
    """
    Affiche plusieurs isosurfaces 3D côte à côte pour visualiser l'évolution.
```

```
    Paramètres
    ----------
    snapshots : list of np.ndarray
        Subset de snapshots à afficher (3 à 4 maximum pour lisibilité)
    iso_level : float
        Niveau de l'isosurface (identique pour tous les snapshots pour
        permettre la comparaison visuelle)
    times_days : list of float, optionnel
        Temps en jours pour les titres de sous-figures

    Notes
    -----
    Utiliser plt.subplots(1, len(snapshots), subplot_kw={'projection': '3d'})
    pour créer une rangée de panneaux 3D côte à côte.
    """
    pass


def downsample_volume(volume: np.ndarray, factor: int = 2) -> np.ndarray:
    """
    Réduit la résolution d'un volume 3D par sous-échantillonnage.

    Utile pour accélérer la visualisation 3D sur des volumes larges.
    Un factor=2 divise chaque dimension par 2 (volume 8x plus petit).

    Paramètres
    ----------
    volume : np.ndarray, shape (X, Y, Z)
    factor : int
        Facteur de sous-échantillonnage (2 ou 4 en pratique)

    Retourne
    --------
    np.ndarray, shape (X//factor, Y//factor, Z//factor)

    Notes
    -----
    Méthode simple : volume[::factor, ::factor, ::factor]
    Méthode avec anti-aliasing (meilleure qualité) :
        skimage.transform.downscale_local_mean(volume, (factor, factor, factor))
    Doc : https://scikit-image.org/docs/stable/api/skimage.transform.html
    """
    pass
```

9. Fichier principal `main.py`

Ce fichier orchestre tout le pipeline. Il ne contient pas de logique métier — il appelle les fonctions des modules précédents dans l'ordre.

python

```python
# main.py

from src.loader import load_nifti, get_voxel_size
from src.preprocessing import (
    normalize_volume, extract_brain_mask,
    build_tissue_masks, get_initial_tumor
)
from src.diffusion_map import build_diffusion_map, rescale_diffusion_for_voxel_size
from src.laplacian import build_laplacian_3d
from src.simulation import build_cn_matrices, run_simulation
from src.visualization import plot_axial_slices, plot_3d_isosurface, plot_3d_evolution


# ——— Paramètres de simulation ———————————————————————————————————————

PATIENT_DIR   = "data/BraTS2021_00001/"
RHO           = 0.018    # taux de prolifération (j⁻¹)
DT            = 1.0      # pas de temps (jours)
N_STEPS       = 300      # nombre de pas → 300 jours de simulation
SAVE_EVERY    = 30       # sauvegarder un snapshot tous les 30 jours
ISO_LEVEL     = 0.15     # niveau de l'isosurface pour la visu 3D


def main():

    # —— Étape 1 : Chargement ——————————————————————————————————————————
    t1,    affine = load_nifti(PATIENT_DIR + "t1.nii.gz")
    flair, _      = load_nifti(PATIENT_DIR + "flair.nii.gz")
    seg,   _      = load_nifti(PATIENT_DIR + "seg.nii.gz")
    voxel_size    = get_voxel_size(PATIENT_DIR + "t1.nii.gz")
    print(f"Volume chargé : {t1.shape}, taille voxel : {voxel_size} mm")

    # —— Étape 2 : Prétraitement ——————————————————————————————————————
    t1_norm    = normalize_volume(t1)
    brain_mask = extract_brain_mask(t1_norm)
    wm, gm, csf = build_tissue_masks(t1_norm, brain_mask)
    c0_3d      = get_initial_tumor(seg)
    print(f"Voxels cerveau : {brain_mask.sum()}, voxels tumeur initiale : {(c0_3d > 0).sum()}")

    # —— Étape 3 : Carte de diffusion ——————————————————————————————————
    D_map = build_diffusion_map(wm, gm, csf)
    D_map = rescale_diffusion_for_voxel_size(D_map, voxel_size[0])

    # —— Étape 4 : Laplacien ——————————————————————————————————————————
    print("Construction du Laplacien (peut prendre quelques minutes) ...")
```

```
    print("Construction du Laplacien (peut prendre quelques minutes)...")
    L = build_laplacian_3d(D_map, brain_mask)
    print(f"Laplacien : matrice {L.shape}, {L.nnz} éléments non nuls")

    # ── Étape 5 : Simulation ──────────────────────────────────────────────
    c0_flat = c0_3d[brain_mask]   # aplatir en 1D pour la simulation
    A, B    = build_cn_matrices(L, DT)

    print(f"Lancement simulation : {N_STEPS} pas de {DT} jour(s)...")
    snapshots = run_simulation(
        c0_flat, A, B, RHO, DT, N_STEPS,
        brain_mask, save_every=SAVE_EVERY
    )

    times = [i * DT * SAVE_EVERY for i in range(len(snapshots))]
    print(f"Simulation terminée. {len(snapshots)} snapshots sauvegardés.")

    # ── Étape 6 : Visualisation ───────────────────────────────────────────
    plot_axial_slices(snapshots, times_days=times)
    plot_3d_isosurface(snapshots[-1], iso_level=ISO_LEVEL, brain_mask=brain_mask)
    plot_3d_evolution([snapshots[0], snapshots[len(snapshots)//2], snapshots[-1]],
                      iso_level=ISO_LEVEL,
                      times_days=[times[0], times[len(times)//2], times[-1]])
    plt.show()


if __name__ == "__main__":
    main()
```

---

10. Organisation des fichiers

Structure recommandée du dépôt Git :

```
projet-tumeur/
│
├── main.py                    ← point d'entrée
│
├── src/
│   ├── __init__.py            ← fichier vide (rend src importable)
│   ├── loader.py              ← étape 1 : chargement NIfTI
│   ├── preprocessing.py       ← étape 2 : masques, condition initiale
│   ├── diffusion_map.py       ← étape 3 : carte D(x)
│   ├── laplacian.py           ← étape 4 : opérateur Laplacien creux
│   ├── simulation.py          ← étape 5 : boucle Crank-Nicolson
│   └── visualization.py       ← étape 6 : coupes 2D et isosurface 3D
│
├── data/
│   └── BraTS2021_00001/       ← données d'un patient (non versionnées)
│       ├── t1.nii.gz
│       ├── t1ce.nii.gz
│       ├── t2.nii.gz
│       ├── flair.nii.gz
│       └── seg.nii.gz
│
├── notebooks/
│   └── exploration.ipynb      ← pour tester les fonctions une par une
│
├── docs/
│   ├── section01_fondements_mathematiques.md
│   └── section03_guide_implementation.md   ← ce fichier
│
├── requirements.txt           ← nibabel numpy scipy matplotlib scikit-image
├── .gitignore                 ← inclure data/ et __pycache__/
└── README.md
```

`.gitignore` recommandé :

```
data/
__pycache__/
*.pyc
.ipynb_checkpoints/
*.egg-info/
```

Conseils pratiques pour démarrer

1. Tester sur un sous-volume d'abord

Avant de lancer la simulation sur le cerveau entier, travailler sur un cube de 50×50×50 voxels centré sur la tumeur. Cela permet de valider chaque fonction en quelques secondes au lieu de plusieurs minutes.

2. Ordre de développement recommandé

Implémenter et tester dans cet ordre : `loader` → `preprocessing` → `diffusion_map` → `laplacian` (le plus difficile) → `simulation` sur petit volume → `simulation` sur volume complet → `visualization`.

3. Valider le Laplacien avant tout

Tester `build_laplacian_3d` sur un cube homogène (D constant partout) : la somme de chaque ligne de la matrice doit être nulle (propriété du Laplacien discret). C'est une vérification rapide et fiable.

4. Commencer avec D constant

Pour la première simulation fonctionnelle, utiliser `D_map = np.full(shape, 0.013)` (D uniforme partout). Cela élimine les sources d'erreurs liées à la segmentation tissu et permet de valider le schéma numérique isolément.

5. Interprétation visuelle

Une simulation correcte montre : une sphère qui grossit et s'étale (au début isotrope si D uniforme), puis une forme asymétrique suivant la matière blanche quand D hétérogène est activé.